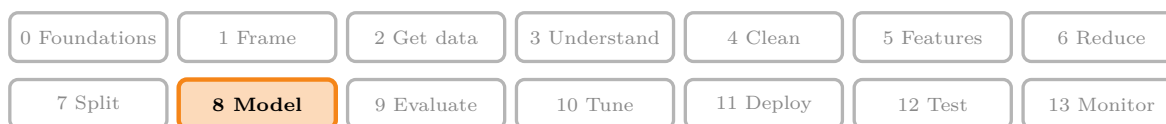


Stochastic Gradient Descent

Contents

1. Overview	2
2. The problem with batch gradient descent	2
3. How stochastic gradient descent works	3
4. Faster progress per epoch	4
5. A noisy path	5
5.1 The zigzag	5
5.2 Advantages of the noise	5
5.3 Disadvantage: jitter at the bottom	6
6. Learning schedules	6
7. SGD in scikit-learn	6
8. When to use which	7
9. Summary	8
10. Key terms	8

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Gradient descent ([Note 57](#)).
- **Compare with:** Batch gradient descent ([Note 58](#)).

1. Overview

Key point: Stochastic gradient descent updates the coefficients after every single random row. Each step is cheap and noisy, so it gets close to the answer in far fewer epochs than batch gradient descent, but it jitters around the minimum.

Batch gradient descent, from the previous Note, uses every row of the training data for each update. **Stochastic gradient descent (SGD)** uses just one row, picked at random, for each update. It is probably the most widely used kind of gradient descent, and the basis of how neural networks are trained.

This Note explains the problem with batch gradient descent, how SGD works, its behaviour, and how to use it in scikit-learn.

2. The problem with batch gradient descent

Key point: Every batch update touches every row and every column. With big data, that is a huge amount of work per step, and the whole dataset must sit in memory.

To update once, batch gradient descent computes the derivative for every coefficient, and each derivative sums over all n rows. So one epoch costs about $n \times m$ multiplications, and the whole run costs about $n \times m \times \text{epochs}$.

With numbers: a dataset of 100,000 rows and 100 columns, trained for 1,000 epochs, needs about

$$100,000 \times 100 \times 1,000 = 10^{10}$$

multiplications, ten billion, just for the derivatives.

Two problems follow:

1. **Speed:** each single step waits for a full pass over the data.
2. **Memory:** the vectorised update needs the whole dataset in memory at once. A dataset larger than the computer's memory cannot be trained this way.

3. How stochastic gradient descent works

Key point: In each epoch, repeat n times: pick one random row, compute the derivatives from that row alone, and update.

The word **stochastic** means random. The algorithm:

1. Start with any coefficients, for example $\beta_0 = 0$ and every $\beta_j = 1$.
2. For each epoch, repeat n times:
 - pick one row i at random;
 - compute its prediction $\hat{y}_i$ and error $y_i - \hat{y}_i$;
 - update every coefficient using the derivatives from that one row.
3. Stop after the chosen number of epochs.

The derivatives are the batch ones with the sum over rows removed: only row i remains.

$$\frac{\partial L}{\partial \beta_0} = -2(y_i - \hat{y}_i) \quad \frac{\partial L}{\partial \beta_j} = -2(y_i - \hat{y}_i) x_{ij}$$

With numbers: if row i has error $y_i - \hat{y}_i = 5$ and $x_{ij} = 0.04$, then $\partial L / \partial \beta_j = -2 \times 5 \times 0.04 = -0.4$, and with learning rate 0.01 the coefficient rises by 0.004.

So an epoch of SGD makes n small updates instead of one big one. On the diabetes data that is 353 updates per epoch.

Python: Stochastic gradient descent from scratch.

```
class MySGDRegressor:
    def __init__(self, learning_rate=0.01, epochs=100, seed=0):
        self.lr, self.epochs = learning_rate, epochs
        self.rng = np.random.default_rng(seed)

    def fit(self, X, y):
        n = X.shape[0]
        self.intercept_ = 0.0, np.ones(X.shape[1])
        for _ in range(self.epochs):
            for _ in range(n):
                i = self.rng.integers(n)          # one random row
                err = y[i] - (X[i] @ self.coef_ + self.intercept_)
                self.intercept_ -= self.lr * (-2 * err)
                self.coef_ -= self.lr * (-2 * err * X[i])
        return self
```

The fixed seed makes the random choice of rows the same on every run, so the results repeat.

Extra: Picking rows at random can choose some rows twice in an epoch and others not at all. A common alternative is to shuffle the rows once per epoch and walk through them in that order, so each row is used exactly once. scikit-learn's `SGDRegressor` does

this (its `shuffle` parameter).

4. Faster progress per epoch

Key point: In 40 epochs, SGD reaches test R^2 0.42 on the diabetes data; batch gradient descent only reaches 0.19.

Figure 1 compares the two on the diabetes data, epoch by epoch.

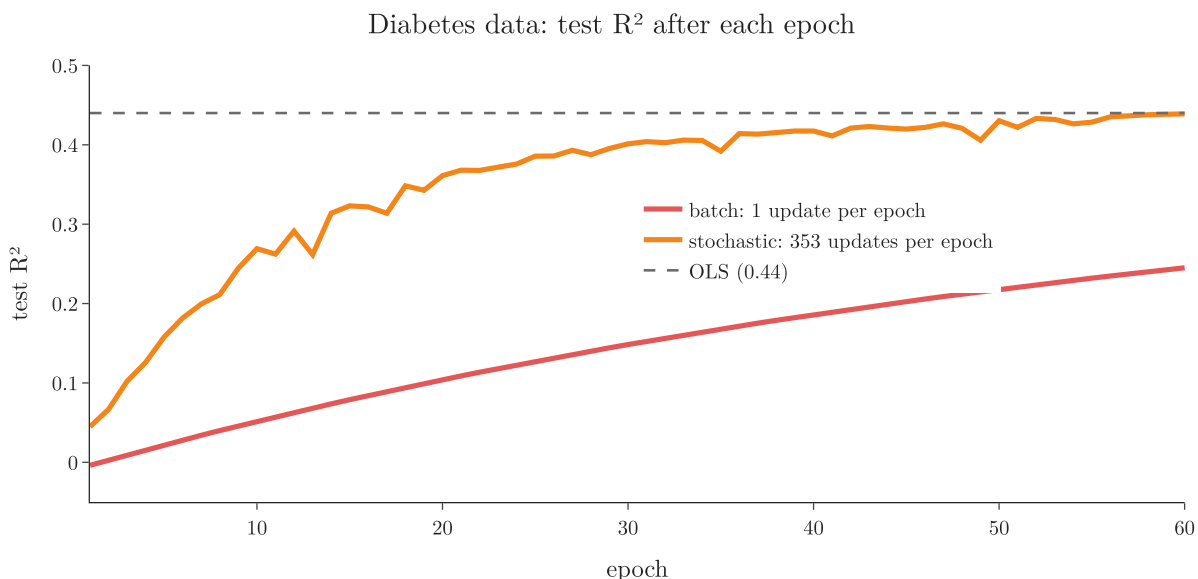


Figure 1: Test R^2 after each epoch: batch vs stochastic

After	Batch (learning rate 0.5)	Stochastic (learning rate 0.01)
10 epochs	0.05	0.27
40 epochs	0.19	0.42
OLS	0.44	0.44

SGD updates 353 times per epoch, batch only once. So per epoch, SGD makes far more progress, even though each of its steps is based on a single row.

Extra: Per epoch, the Python loop above is slower than batch gradient descent (0.1 seconds for 40 epochs against a few milliseconds), because a Python loop over rows is slow and batch uses one fast matrix product. Real SGD implementations, such as scikit-learn's (written in compiled code) or deep-learning libraries, do not have this problem. The advantage of SGD is the number of passes over the data, which is what matters when data is huge or does not fit in memory.

5. A noisy path

Key point: One row is a rough estimate of the whole gradient, so SGD's path zigzags. That makes it fast and able to escape local minima, but it never settles exactly at the minimum.

5.1 The zigzag

Key point: Each step points roughly, not exactly, downhill.

The gradient from one row is a noisy estimate of the gradient from all rows. Figure 2 (left) shows the consequence on the 100-point example: batch gradient descent (red) walks straight down the bowl, while SGD (orange) wanders but reaches the bottom area within one epoch.

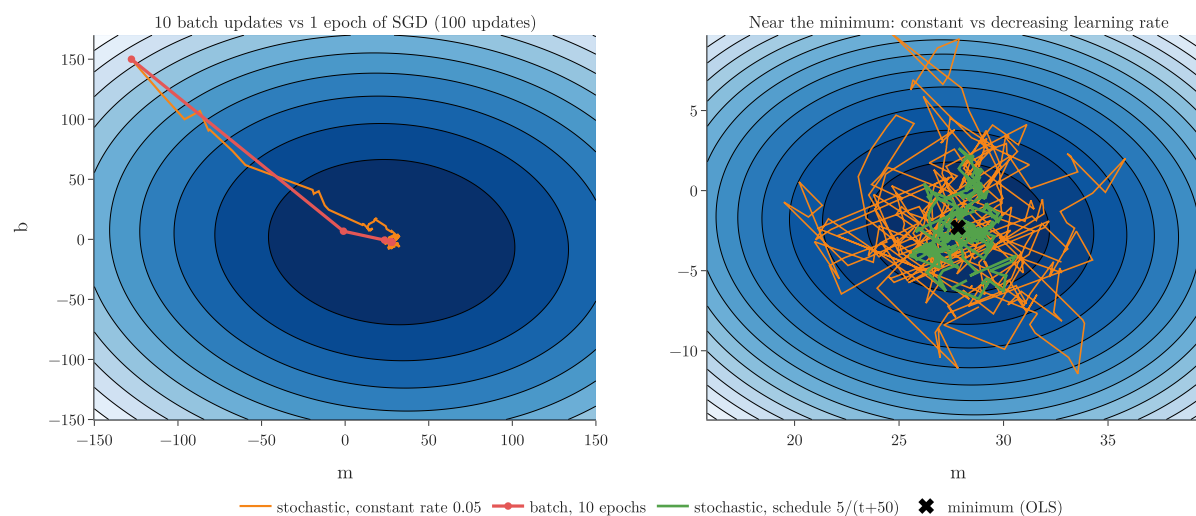


Figure 2: Batch vs stochastic paths, and the effect of a learning schedule

5.2 Advantages of the noise

Key point: The noise speeds SGD up on large data and can shake it out of a local minimum.

- **Large data:** SGD needs only one row at a time in memory, and it gets close to the answer in few passes.
- **Non-convex losses:** with a loss that has local minima (the gradient descent Note), batch gradient descent settles in whichever dip it reaches first. SGD's random jumps can carry it out of a shallow local minimum and on towards the global one. This is one reason it is used for neural networks.

5.3 Disadvantage: jitter at the bottom

Key point: With a constant learning rate, SGD keeps jumping around the minimum instead of stopping at it.

Near the minimum, every new row still pushes the coefficients in its own direction. So SGD keeps bouncing around the best values (Figure 2, right, orange) and never stops exactly there. Running it twice with different random rows also gives slightly different answers.

6. Learning schedules

Key point: Making the learning rate shrink over time lets SGD move fast at first and settle down near the end.

The fix for the jitter is a **learning schedule**: a learning rate that decreases as training goes on. Big steps early on cover distance quickly; small steps later let SGD settle close to the minimum.

A common schedule is

$$\eta_t = \frac{t_0}{t + t_1}$$

where t counts the updates done so far, and t_0 and t_1 are constants. With $t_0 = 5$ and $t_1 = 50$:

Update t	0	100	1,000	10,000
Learning rate	0.1	0.033	0.0048	0.0005

In Figure 2 (right), the scheduled run (green) stays much closer to the minimum than the constant-rate run (orange): over the last 200 updates its average distance from the best values is 2.1, against 4.7.

Extra: Gradually lowering the learning rate is sometimes called *simulated annealing*, by analogy with cooling metal slowly so that it settles into a stable state.

7. SGD in scikit-learn

Key point: SGDRegressor trains linear regression with stochastic gradient descent; `learning_rate` and `eta0` set the schedule.

Python: scikit-learn's SGDRegressor.

```
from sklearn.linear_model import SGDRegressor
```

```
reg = SGDRegressor(max_iter=100, learning_rate="constant",  
                    eta0=0.01, random_state=0)
```

```
reg.fit(X_train, y_train)
reg.score(X_test, y_test)    # R2 0.43
```

The main parameters:

Parameter	Meaning
<code>max_iter</code>	the maximum number of epochs
<code>eta0</code>	the starting learning rate
<code>learning_rate</code>	the schedule: "constant", "invscaling" (the default, $\eta_0/t^{0.25}$), "optimal" or "adaptive"
<code>tol</code>	stop early when the loss stops improving by at least this much
<code>random_state</code>	fixes the random order of rows

On the diabetes data, "constant" with `eta0=0.01` reaches test R^2 0.43 in 97 epochs. The default "invscaling" schedule shrinks the rate too fast for this data and stops at 0.16 after 100 epochs. Schedules need tuning like any other hyperparameter.

Extra: scikit-learn may print a `ConvergenceWarning` when `max_iter` runs out before its stopping rule (`tol`) is met. It is a hint to raise `max_iter` or the learning rate, not an error.

8. When to use which

Key point: Batch for small data and smooth convex losses; SGD for very large data, data that does not fit in memory, and non-convex losses.

	Batch	Stochastic
Rows per update	all	one
Path	smooth, exact gradient	noisy, zigzags
Progress per epoch	slow	fast
Memory	whole dataset	one row
Final answer	settles at the minimum	jitters around it (use a schedule)
Local minima	can get stuck	can escape

Mini-batch gradient descent, the next Note, sits between the two and is what most practice uses.

9. Summary

- Batch gradient descent needs about $n \times m$ operations per update: too slow and too memory-hungry for big data.
- SGD updates after each random row: n updates per epoch, each using one row's derivatives.
- On the diabetes data, 40 epochs of SGD reach test R^2 0.42, against 0.19 for batch.
- The noise makes SGD fast and able to escape local minima, but it jitters at the bottom.
- A learning schedule (shrinking rate) lets it settle; `SGDRegressor` offers several.

10. Key terms

Term	Meaning
Stochastic	Involving randomness
Stochastic gradient descent (SGD)	Gradient descent that updates the coefficients after each single random row
Learning schedule	A rule that changes the learning rate during training, usually shrinking it
SGDRegressor	scikit-learn's linear regression trained with stochastic gradient descent
eta0	The starting learning rate in SGDRegressor
max_iter	The maximum number of epochs in SGDRegressor
Simulated annealing	Lowering the learning rate gradually so the search settles down